



Bucharest University of Economic Studies
Faculty of Cybernetics, Statistics and Economic Informatics
Economic Informatics Program

Raspberry Pi Platform for Security Analysis of Short-Range Communication Protocols

Bachelor Thesis

Coordinator

Prof. univ. dr. Catalin Boja

Graduate

Florin Melinte

Bucharest 2026

Declaration of Originality and Responsible Use of Artificial Intelligence

I hereby declare that this bachelor's/master's thesis is my own original work and that the results, analyses, interpretations, and conclusions presented herein are entirely the result of my own intellectual effort, except where explicit reference is made to the work of other authors.

I confirm that all materials, ideas, data, figures, and information obtained from external sources, including books, journal articles, conference papers, websites, and other publications, have been properly acknowledged and cited in accordance with academic standards and are included in the bibliography.

Where Generative Artificial Intelligence (AI) tools were used during the preparation of this thesis, such use was limited to permitted support activities (such as language revision, translation, idea generation, or other disclosed purposes), was carried out in accordance with the university's guidelines, and has been transparently documented where required. I remain fully responsible for the content, accuracy, originality, methodology, analysis, and conclusions of this thesis.

I understand that any false declaration, plagiarism, misrepresentation of authorship, or undisclosed misuse of Generative AI tools may constitute a violation of academic integrity regulations and may result in disciplinary or legal consequences.

Table of Contents

1. Introduction.....	1
2. Domain Analysis	3
3. Technologies, Tools and Methods	8
3.1 Hardware Platform	8
3.2 Software Stack and External Tools	9
4. Solution Architecture and Design	12
4.1 System Architecture	12
4.2 Attack Module Design.....	13
5. Implementation Details	15
5.1 Development Challenges	15
5.2 Attack Implementation Highlights	16
5.3 Key Technical Decisions	19
6. Flipper Zero vs My Implementation	21
7. Conclusions and Future Work	24
References and Bibliography.....	26
Annexes	28

1. Introduction

The past decade has seen wireless communication become the dominant medium for personal and enterprise networking. WiFi connects billions of devices to the internet; Bluetooth Low Energy is embedded in everything from medical monitors to smartwatches; phishing campaigns delivered over these same networks now account for the majority of data breaches worldwide. As these technologies have spread, the need to train security professionals who understand how they can be exploited, and how to defend against those exploits, has grown accordingly. Yet the tools and platforms available for hands-on security education remain either prohibitively expensive, closed-source, or so complex that beginners cannot effectively learn from them.

Cybersecurity education faces a persistent gap between theory and practice. University courses cover WiFi protocol layers, Bluetooth pairing mechanisms, and encryption algorithms in detail, and it is highly beneficial for students to complement this theoretical knowledge with practical application. This platform helps students gain hands-on experience executing real attacks in a controlled environment. For example, when they learn that deauthentication frames exploit unprotected WiFi management frames, they should be able to actively send one and watch a device disconnect to fully grasp the concept.

Professional tools exist, but they create barriers rather than removing them. Performing a simple WiFi security test requires chaining together airmon-ng, airodump-ng, and mdk4, three different tools with different interfaces and parameters. Commercial platforms like Metasploit Pro and Nessus cost \$1,000–10,000+ per year and hide their implementation behind proprietary code. The Flipper Zero, a portable hacking device that inspired this project, demonstrated that a small device can perform meaningful WiFi and Bluetooth attacks, but it costs \$300+, is closed source, and prevents students from inspecting how its attacks actually work.

This thesis presents the design and implementation of an open-source security research platform that addresses these problems. The platform runs on a Raspberry Pi Zero 2 W (~\$15) with a TP-Link WiFi adapter (~\$20), bringing the total cost to approximately \$35 per station, compared to \$300+ for a Flipper Zero or \$5,000+ for professional platforms. It provides 16 security attacks across four domains: 8 WiFi attacks (network scanning, beacon spoofing, deauthentication, packet capture, ESSID brute-force, network flooding, HTTP DoS, local network DoS), 5 BLE attacks (device scanning, AirPods spam, Android spam, Apple device spam, name spoofing), 1 Bluetooth Classic attack (L2CAP DoS), and 2 phishing attacks (Facebook and Google login replicas with Cloudflare Tunnel support). All attacks are accessible through a unified, colour-coded terminal menu system.

The project set out to achieve six objectives:

1. Create a unified platform that abstracts individual security tools behind a single menu-driven interface.
2. Demonstrate practical vulnerabilities through executable code that produces real, observable effects.
3. Design an intuitive interface with colour-coded output and input validation for beginners.
4. Implement a modular architecture where each attack is an independent Python module.
5. Provide a fully transparent codebase that students can inspect, modify, and learn from.
6. Document the implementation thoroughly for reproducibility and community contribution.

The remainder of this thesis is structured as follows. Chapter 2 provides a domain analysis of the security concepts the platform addresses, covering WiFi protocol vulnerabilities, Bluetooth Low Energy advertisement exploits, phishing mechanics, and the educational gap this project fills. Chapter 3 describes the hardware and software technologies chosen and explains the rationale behind each choice. Chapter 4 presents the system architecture and design patterns that make the platform consistent and extensible. Chapter 5 details the most technically interesting implementation decisions and the challenges encountered during development. Chapter 6 compares the platform against the Flipper Zero device that inspired it, analysing where each excels and why the differences exist. Chapter 7 states the conclusions and proposes directions for future work.

2. Domain Analysis

This chapter explains the security concepts behind the attacks implemented in the platform. Understanding how WiFi and Bluetooth protocols work, and where they are vulnerable, is essential for understanding why the attacks succeed and how defences can be deployed. The following sections cover the relevant protocol mechanics first, then identify the specific weaknesses the platform exploits, and finally outline the defences that can mitigate each attack class. The final section ties these together by examining the educational landscape, contrasting my approach against existing tools and platforms.

It is important to emphasise that all attacks described in this thesis were conducted exclusively on equipment owned by the author or on networks for which explicit authorisation was obtained. The platform is designed as an educational tool; its purpose is to make the mechanics of well-known vulnerabilities visible and understandable, not to enable malicious use. The ethical and legal framework governing responsible use is discussed further in Annex B.

WiFi networks use the IEEE 802.11 standard and operate primarily on 2.4 GHz and 5 GHz frequency bands. The protocol defines three types of frames: management frames (for network discovery and connection management), control frames (for flow control), and data frames (carrying actual network traffic).

Management frames are the foundation of most WiFi attacks in this platform. They include beacon frames, which access points broadcast every ~100ms to announce their presence; probe requests and responses, which devices use to discover networks; and authentication/deauthentication frames, which control joining and leaving a network. The critical vulnerability is that management frames are not encrypted or authenticated in WPA2, which remains the dominant WiFi security standard. Any device with a monitor-mode WiFi adapter can forge management frames and inject them into the network.

This vulnerability enables several attack categories. Deauthentication attacks send forged disconnect frames that cause devices to drop their WiFi connections, the target sees what appears to be a legitimate disconnect command and complies. Beacon spoofing creates fake WiFi networks that appear in nearby devices' network lists by broadcasting arbitrary beacon frames. SSID enumeration discovers hidden networks by sending probe requests with common names and listening for responses. Packet capture records all WiFi traffic to files that can be analysed in Wireshark.

The WPA2 authentication process follows the four-way handshake protocol. When a client device attempts to connect to an access point, the following sequence occurs: (1) the access point sends a random nonce (ANonce) to the client; (2) the client generates its own nonce

(SNonce), derives the Pairwise Transient Key (PTK) from both nonces and the Pre-Shared Key, then sends the SNonce alongside a Message Integrity Code (MIC) to the AP; (3) the AP derives the same PTK independently and verifies the MIC, then sends the Group Temporal Key (GTK) encrypted with the PTK; (4) the client installs the keys and sends a confirmation. The handshake establishes encrypted communication without transmitting the password in plain text. However, it also creates the attack surface for offline dictionary attacks: if an attacker captures the handshake frames (all four messages are transmitted over the air in the clear), they can attempt to derive the PTK offline by trying candidate passwords until the MIC verification succeeds.

WiFi security has evolved through several generations, each introduced in response to vulnerabilities in the previous standard:

Table 2.1. Evolution of the WiFi security standards

Standard	Year	Encryption	Key Vulnerability	Status
WEP	1997	RC4 (40/104-bit)	Key reuse, IV collisions allow key recovery in minutes	Deprecated
WPA	2003	TKIP (RC4-based)	Weak MIC (Michael), TKIP crackable under conditions	Deprecated
WPA2	2004	AES-CCMP (128-bit)	Unprotected management frames, PMKID offline cracking	Current (dominant)
WPA3	2018	AES-CCMP + SAE	Protected Management Frames; SAE resists offline attack	Adoption growing

WPA3 adoption remains limited; most home and business routers still use WPA2, which is why the deauthentication and spoofing attacks in this platform remain effective on the majority of modern networks. The Protected Management Frames (PMF) feature introduced in WPA3 and optionally available in WPA2 would block deauthentication attacks by cryptographically authenticating management frames, making them impossible to forge.

Additionally, devices on the same local network are vulnerable to network-layer flooding attacks. My platform demonstrates this using ARP scanning to discover devices, followed by SYN packet flooding with hping3. In testing, this caused a target iPhone to freeze completely within seconds.

Defence mechanisms include enabling WPA3 with Protected Management Frames, implementing network segmentation, deploying intrusion detection systems, and using rate limiting for connection requests.

Beyond WiFi, the platform also targets Bluetooth, which presents a distinct but equally significant set of vulnerabilities rooted in its advertisement-based communication model.

Bluetooth Low Energy (BLE), introduced in Bluetooth 4.0 (2010), is designed for low-power devices like fitness trackers, wireless earbuds, and smart home sensors. BLE devices communicate their presence by broadcasting advertisement frames, short packets (up to 31 bytes) on three dedicated advertising channels. These advertisements contain the device address, device name, manufacturer-specific data (identified by a 2-byte company identifier assigned by the Bluetooth SIG), service UUIDs, and TX power level.

A BLE advertisement packet has a tightly defined structure. The payload consists of a sequence of AD (Advertising Data) structures, each beginning with a length byte, followed by a type byte, followed by the data. The manufacturer-specific type (0xFF) carries a two-byte company identifier registered with the Bluetooth SIG, followed by up to 27 bytes of vendor-defined payload. For example, a genuine Apple AirPods advertisement begins with 0xFF 0x4C 0x00 (type + Apple's little-endian company ID), followed by the proximity pairing model code, status flags, and battery level bytes for the left ear, right ear, and charging case. Because no signature or authentication token protects this structure, any device can transmit the exact same byte sequence and the recipient's operating system will react identically to receiving it from a real AirPods device.

The fundamental vulnerability exploited by this platform is that BLE advertisements are unauthenticated. There is no mechanism to verify that an advertisement actually comes from the device it claims to be. Any device with a Bluetooth adapter can broadcast advertisements using any manufacturer ID and any device type. This enables my AirPods spam attack (broadcasting fake Apple AirPods advertisements using Apple's company ID 0x004C to trigger iOS pairing popups), my Android spam attack (targeting Google and Samsung devices using their respective company identifiers), and my Apple device spam attack (spoofing AirTags, Apple TVs, and HomePods by using the correct proximity action payload codes for each device type). The protocol accepts these advertisements because stronger authentication would increase power consumption, contradicting BLE's low-power design goals, and because the original BLE specification predates widespread deployment of these types of spam attacks.

My platform's BLE device scanner leverages this advertisement system constructively: it discovers nearby devices and identifies their manufacturers using a YAML database of company identifiers, estimates distance from signal strength (RSSI), and tracks when devices appear and disappear.

Bluetooth Classic (the older, higher-bandwidth protocol) uses L2CAP (Logical Link Control and Adaptation Protocol) for connection management. When a device receives an L2CAP connection request, it allocates resources to handle it. My L2CAP DoS attack exploits this by sending rapid echo requests to exhaust the target's connection handling capacity. Modern

Bluetooth implementations mitigate this with rate limiting, which itself is an educational lesson: protocols evolve to address known attacks.

Defence mechanisms include disabling unnecessary Bluetooth discoverability, keeping firmware updated to enable rate limiting, and treating device names and advertisements as untrustworthy identifiers.

The third attack domain shifts from protocol-level exploitation to social engineering, targeting human behaviour rather than network vulnerabilities.

Phishing is one of the most effective attack vectors in cybersecurity; by some estimates, it is involved in over 30% of data breaches. My platform includes two phishing modules (Facebook and Google login replicas) that demonstrate why this attack remains so effective.

The attack works by hosting a replica of a legitimate login page on a local HTTP server, capturing credentials when the victim submits the form, logging them with a timestamp and IP address, and redirecting the victim to the real website so they may not realise anything happened. The HTML/CSS for these replicas is embedded directly in the Python source files, making the entire attack transparent and inspectable.

To make phishing pages accessible beyond the local network, the platform integrates Cloudflare Tunnel, a tool that creates an outbound connection from the Raspberry Pi to Cloudflare's edge network, generating a public HTTPS URL without requiring port forwarding, a public IP address, or SSL certificate management. This demonstrates how modern cloud infrastructure eliminates traditional barriers to hosting attacks, an important awareness point for security education.

Defence mechanisms include verifying URLs before entering credentials, using password managers that detect domain mismatches, enabling multi-factor authentication, and deploying organisational security awareness training.

The attacks described above are not new. They have been documented, demonstrated at security conferences, and reported in academic literature for over a decade. What is lacking is not knowledge of these vulnerabilities, but accessible, transparent tools that allow students to study them in practice rather than only in theory.

Existing tools fall into three categories, each with significant educational limitations.

Individual command-line tools such as aircrack-ng, hcitool, and mdk4 are powerful but require expert knowledge to use effectively. A student who wants to understand a deauthentication attack must first learn to configure a monitor-mode interface, then understand airodump-ng's output format, then construct the correct mdk4 command, and finally interpret the results, all before they can observe the attack working. The tools do not explain themselves, and the workflow for combining them is not documented in any single

place. The learning curve can take days or weeks before a beginner achieves their first successful test.

Commercial platforms such as Metasploit Pro, Burp Suite Professional, and enterprise network assessment tools are optimised for efficiency rather than education. Their source code is closed, so students interact with the tool's outputs rather than understanding the underlying implementation. Licensing costs (\$1,000–\$10,000+ per year) make individual student access impractical, and the feature sets are so comprehensive that beginners are overwhelmed before they can focus on the specific concept they are studying.

The Flipper Zero portable device sits in a third category: it is affordable relative to enterprise tools, has a polished user experience, and can perform a wide range of attacks. However, it is still closed source (its official firmware does not expose implementation details), it costs \$300+, and its firmware upgrade path is controlled by the manufacturer. Students can use it to demonstrate an attack but cannot read the code that makes it work. Furthermore, at \$300+ per unit, deploying 30 lab stations for a university course would cost approximately \$9,000, a budget that most cybersecurity programmes cannot justify for a single piece of hardware.

My platform addresses all three limitations simultaneously. It is open source and written in readable Python, so a student can open any attack file and follow exactly how the attack is constructed. It integrates the necessary command-line tools behind a unified menu, removing the barrier of needing to know which tools to combine and in what order. It runs on hardware costing approximately \$35 per station, making lab-scale deployment feasible. And because every component is documented, it supports reproducible science: another researcher can replicate every experiment described in this thesis using the same code and the same hardware.

3. Technologies, Tools and Methods

This chapter details the hardware, software, and tools that form the foundation of the platform. All components were chosen to balance educational accessibility, affordability, and practical capability.

Three criteria guided every selection decision. First, **affordability**: the combined hardware cost must remain low enough for universities to deploy multiple lab stations without a prohibitive budget. This ruled out professional-grade network analysis hardware and limited Python library choices to those with manageable memory footprints. Second, **Linux compatibility**: WiFi monitor mode (the capability that makes packet capture and frame injection possible) is supported on Linux but not on Windows or macOS at the operating system level. This anchored the platform to Debian Linux and shaped which tools were available. Third, **educational clarity**: where two tools could accomplish the same task, the one whose operation is more transparent and whose output is easier to interpret was preferred. For example, calling `airodump-ng` and reading its comma-separated output teaches students about the data fields that WiFi scanning exposes, whereas using a higher-level wrapper library would hide that structure.

Section 3.1 describes the hardware components and explains the trade-offs behind each choice. Section 3.2 covers the full software stack — the operating system, Python libraries, and external command-line tools — and explains the rationale behind each choice.

3.1 Hardware Platform

The platform runs on two hardware components:

Raspberry Pi Zero 2 W, the main computing unit. It features a quad-core ARM Cortex-A53 CPU at 1.0 GHz, 512 MB RAM, integrated WiFi 802.11b/g/n (2.4 GHz), and Bluetooth 4.2 BLE. Its compact form factor (65×30×5mm) and ~\$15 cost make it suitable for large-scale classroom deployment. The built-in Bluetooth adapter handles all BLE and Bluetooth Classic attacks without additional hardware. The main constraint is the 512 MB RAM, which limits memory-intensive operations, this affected design decisions discussed in Chapter 5.

TP-Link Archer T2U PLUS, an external USB WiFi adapter providing the capabilities the Pi's built-in WiFi cannot. Its Realtek RTL8821AU chipset supports monitor mode (required for packet capture and injection) and dual-band operation (2.4 GHz and 5 GHz). Monitor mode allows the adapter to capture all 802.11 frames in range, not just those addressed to it, this is essential for passive scanning, deauthentication attacks, and beacon spoofing. The adapter costs ~\$15-20 and uses an open-source community driver installed via DKMS

(Dynamic Kernel Module Support), which automatically recompiles the driver when the Linux kernel updates.

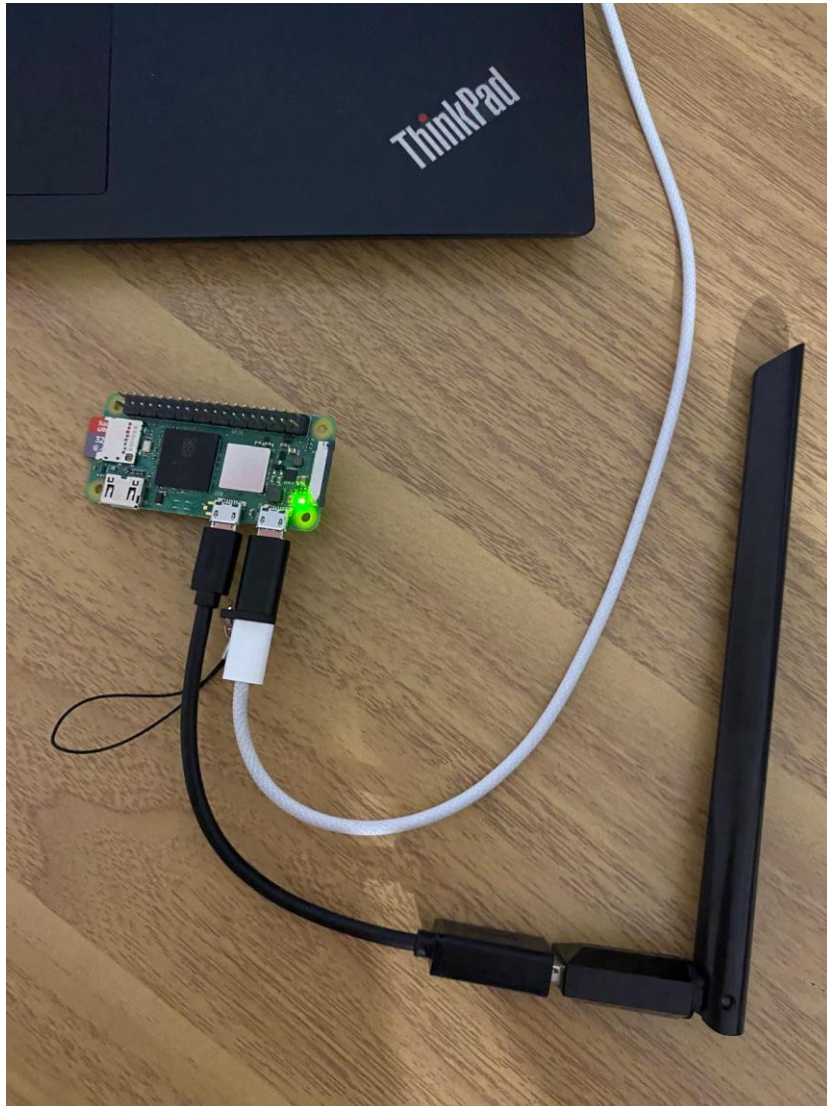


Figure 3.1. Complete hardware setup

3.2 Software Stack and External Tools

The platform runs on Raspberry Pi OS (Debian-based Linux), chosen for its native WiFi monitor mode support, package management via apt, and extensive driver availability. Windows and macOS were not viable because they lack native monitor mode support for WiFi adapters.

The application is written in Python 3, selected for its readability and widespread use in computer science education. Three external Python libraries are used:

- **bleak**, an asynchronous BLE scanning library. Used in the BLE device scanner to discover nearby devices, parse advertisement data, and measure signal strength. Its

async interface (using Python's `asyncio`) allows concurrent scanning of multiple devices without blocking.

- **PyYAML**, a YAML parser used to load the BLE company identifier database (a file mapping manufacturer IDs to company names, e.g., `0x004C` → Apple).
- **PyBluez** (`bluetooth._bluetooth`), provides direct access to the Linux Bluetooth HCI socket layer. Used in the BLE spam attacks (AirPods, Android, and Apple device spam) to send raw HCI commands for BLE advertisement broadcasting.

The Python standard library provides the remaining functionality: `subprocess` for launching external tools, `asyncio` for concurrent BLE operations, `threading` for the multi-threaded HTTP DoS attack and phishing server, `http.server` for hosting phishing pages, and `socket` for network validation.

External Tools

The platform delegates protocol-level operations to compiled external tools rather than implementing them in Python. This is a deliberate design choice: these tools are written in C and handle packet processing efficiently within the Pi's memory constraints. Python serves as the orchestrator, building commands, streaming output line-by-line to the user, and handling interaction. This approach also teaches students how real security workflows integrate multiple specialised tools, which is itself an important skill.

Table 3.1. External command-line tools integrated by the platform

Tool	Purpose	Used By	Why this tool
aircrack-ng suite	WiFi security toolkit: airodump-ng for scanning, airmon-ng for monitor mode	Network scanner, packet capture	Industry-standard suite; airodump-ng's CSV output is well-documented and easy to parse programmatically
mdk4	WiFi frame injection: beacon spam, deauthentication	Beacon broadcast, AP flood, deauth, ESSID bruteforce	Successor to mdk3; supports multiple frame types via a single consistent CLI; actively maintained
hping3	Network packet crafting and SYN flooding	Local network DoS	Supports TCP/UDP/ICMP/RAW-IP at the packet level; SYN flood mode is a single flag (<code>--syn --flood</code>)
arp-scan	ARP-based local network device discovery	Local network DoS (device discovery phase)	Faster and more reliable than <code>nmap -sn</code> for LAN enumeration; output format is simple to parse
bluez /	Linux Bluetooth stack	BLE spam, name	BlueZ is the official Linux

Tool	Purpose	Used By	Why this tool
btmgmt / l2ping	management and L2CAP packet flooding	spoofing, adapter configuration, L2CAP DoS	Bluetooth stack; btmgmt provides direct adapter control; l2ping sends flood-mode L2CAP echo requests
cloudflared	Cloudflare Tunnel client for public URL generation	Phishing modules	Generates a public HTTPS URL with no port forwarding, no static IP, and no SSL certificate setup; free tier sufficient for demonstrations

All system packages are installed via `apt-get` from standard Debian repositories. The complete installation procedure, including the WiFi driver compilation steps, is documented in Annex A.

4. Solution Architecture and Design

This chapter describes the architecture of the platform, how the code is organised, how modules interact, and the design patterns that make the system consistent and extensible.

A guiding principle during design was that the architecture should itself be educational. A student reading the codebase should be able to identify clear boundaries between responsibilities, understand why each component exists, and see how adding a new attack would integrate into the existing structure without modifying unrelated code. This led to two central decisions: a strict layered architecture that separates user interaction from attack logic from system calls, and a uniform interface pattern that every attack module implements regardless of its domain. Section 4.1 describes the layered architecture and its three tiers, including the console module that implements the presentation layer. Section 4.2 explains the attack module design pattern and how it is applied across all four attack domains.

4.1 System Architecture

The application follows a layered architecture with three distinct layers:

Presentation Layer, handles all user interaction. The UI module (`ui/console.py`) provides colour-coded output functions: `iprint()` for informational messages in light blue, `wprint()` for warnings in yellow, `eprint()` for errors in red, and `sprint()` for success messages in green. The `AttackSuite` class in `main.py` manages the three-level menu system (main menu → attack category → specific attack) and routes user choices to the correct attack module.

Application Layer, contains the attack logic. Each of the 16 attacks is implemented as an independent Python class in one of four packages: `wifi/` (8 attacks), `ble/` (5 attacks), `bt/` (1 attack), and `phishing/` (2 attacks). Every attack class follows the same interface pattern: validate parameters → execute the attack → display results. This consistency means adding a new attack requires writing one class and adding one entry to the menu dictionary.

System Layer, interfaces with the operating system and external tools. WiFi attacks use `subprocess.Popen()` to launch tools like `mdk4` and `airodump-ng`, streaming their output line-by-line to the user in real-time. BLE attacks use the `bleak` library's async interface to communicate with the Bluetooth hardware. Phishing attacks use Python's `http.server` to host fake login pages and optionally launch `cloudflared` as a subprocess for public URL generation.

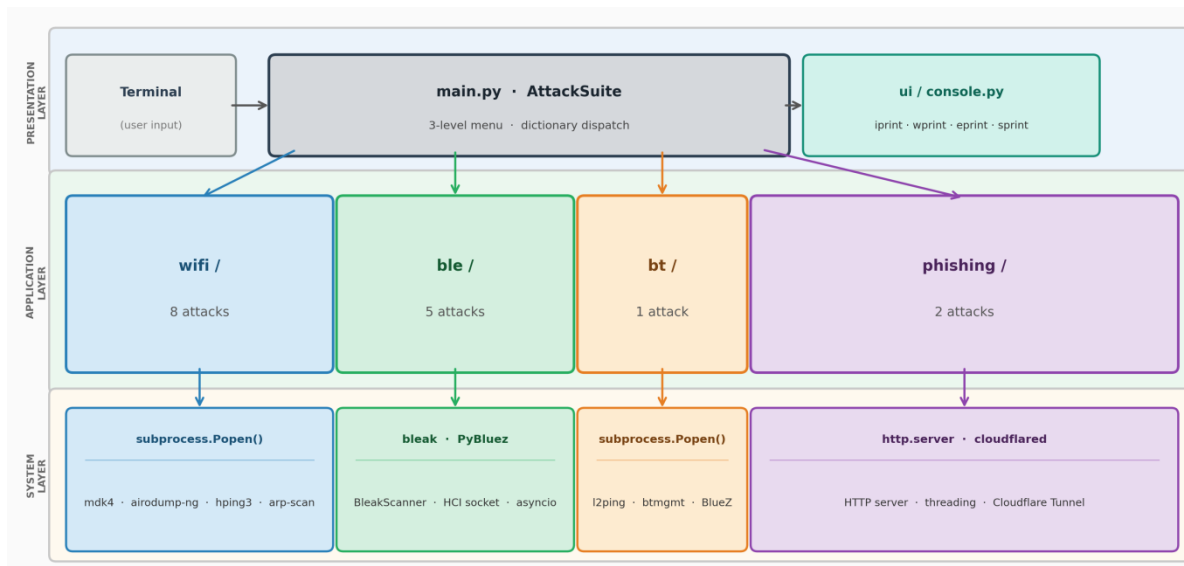


Figure 4.1. System architecture: three-layer component view

The advantage of this separation is that each layer can change independently. Modifying the UI colours only touches `console.py`. Adding a new WiFi attack only adds a file to `wifi/` and a menu entry in `main.py`. The external tools remain untouched.

Console and Output Module

The `ui/console.py` module centralises all terminal output formatting. It defines ANSI escape code constants for colours (RED, GREEN, YELLOW, BLUE, CYAN, etc.) and provides wrapper functions that all attack modules use instead of calling `print()` directly.

The message type system creates a visual hierarchy: blue for status information, yellow for warnings, red for errors, green for success, and cyan for data output during attacks. This consistency means users can scan output quickly, spotting a red line immediately signals a problem, and green confirms success. The `cinput()` function provides coloured input prompts that maintain the visual style throughout the interaction.

The module also provides `clear()` for screen clearing and `print_banner()` for section headers. By keeping all formatting in one file, any visual change — colour scheme, prefix format, banner style — requires editing only `console.py`; the 16 attack modules remain untouched.

4.2 Attack Module Design

All attack modules follow a consistent design pattern that serves both functional and educational purposes.

Standard Attack Pattern:

Every attack class follows the validate → execute → display flow:

1. **Validate**, check that prerequisites are met (WiFi interface exists and is in monitor mode, target IP format is valid, Bluetooth adapter is available).
2. **Execute**, run the attack by calling the appropriate external tool via subprocess or library API. Output is streamed to the user in real-time through the UI module.
3. **Display**, show results (success/failure, packets sent, devices found, credentials captured).

This pattern is implemented differently depending on the attack domain:

WiFi attacks use the subprocess integration pattern. The attack class builds a command (e.g., `['mdk4', interface, 'd', '-E', ssid]`), launches it with `subprocess.Popen()`, and reads output line-by-line in a loop. `KeyboardInterrupt` (Ctrl+C) is caught to terminate the subprocess cleanly. This pattern is the same across all 8 WiFi attacks, only the command and parameters differ.

BLE attacks use Python's `asyncio` for concurrent device handling. The `bleak` library provides `BleakScanner` which calls a detection callback for each advertisement received. The scanner runs in an async event loop, allowing the display table to update in real-time while new devices are still being discovered.

Phishing attacks use `threading` to run the HTTP server in a background thread while the main thread manages the Cloudflare Tunnel subprocess and user interaction. When a victim submits the login form, the `do_POST()` handler captures the credentials, logs them to a file, and returns an HTTP 302 redirect to the real website.

Menu Handler Dispatch:

The main menu uses a dictionary-based dispatch pattern to route user choices to attack methods:

```
action_map = {
    '1': self.action_beacon_broadcast,
    '2': self.action_ap_network_flood,
    '3': self.action_network_scanner,
    ...
}
```

This approach is more scalable and maintainable than if-elif chains: adding a new attack requires only one dictionary entry in the action map and one new method on the class. Nothing else in the menu system changes. The same pattern is replicated in all four category menus (WiFi, BLE, Bluetooth, and Phishing), so a developer familiar with one menu can immediately understand all others. This is an example of the broader architectural philosophy: consistency across the codebase reduces the cognitive load for anyone reading or extending it.

5. Implementation Details

This chapter covers how the attacks were built in practice, the development challenges encountered, the most interesting implementation details across each attack domain, and the key technical decisions that shaped the final platform. Section 5.1 describes the development process and the obstacles encountered during it. Section 5.2 highlights the most technically interesting attack implementations, with enough detail to understand the mechanisms without reproducing the full source listings (those are available in the accompanying repository). Section 5.3 explains the three architectural decisions that had the largest impact on the overall design.

5.1 Development Challenges

Development followed an iterative approach: I started with basic WiFi attacks (beacon broadcast and network scanning) to establish the subprocess integration pattern, then expanded to more complex attacks once the pattern was proven. All development and testing happened directly on the Raspberry Pi Zero 2 W, with code transferred via WinSCP.

Monitor Mode Setup. WiFi attacks require the adapter to be in monitor mode, but the setup varies by driver version and Linux distribution. I added a `verify_monitor_mode()` function to every WiFi attack class that checks `iwconfig` output before execution, preventing attacks from running on misconfigured interfaces and giving users clear error messages.

Driver Compatibility. The TP-Link adapter's RTL8821AU driver is not included in the default Linux kernel. It must be compiled from source using an open-source community driver. I used DKMS to ensure the driver automatically recompiles when the kernel updates, preventing the common problem of adapters stopping to work after system upgrades.

Memory Constraints (512 MB RAM). The Pi's limited memory directly affected feature decisions. I originally planned to include PMKID password cracking using hashcat, but this proved impossible: hashcat is designed for GPU acceleration (the Pi has none), requires 2-4 GB of RAM for wordlist operations, and cannot be compiled for ARM without significant modification. Rather than shipping a broken feature, I removed it entirely. Users who capture handshakes with my packet capture tool can transfer the files to a desktop machine for cracking, which is how professional penetration testers work in practice. The memory constraint also motivated the subprocess pattern: external tools like `mdk4` and `hping3` are compiled C programs that manage memory efficiently, while Python acts purely as the orchestrator.

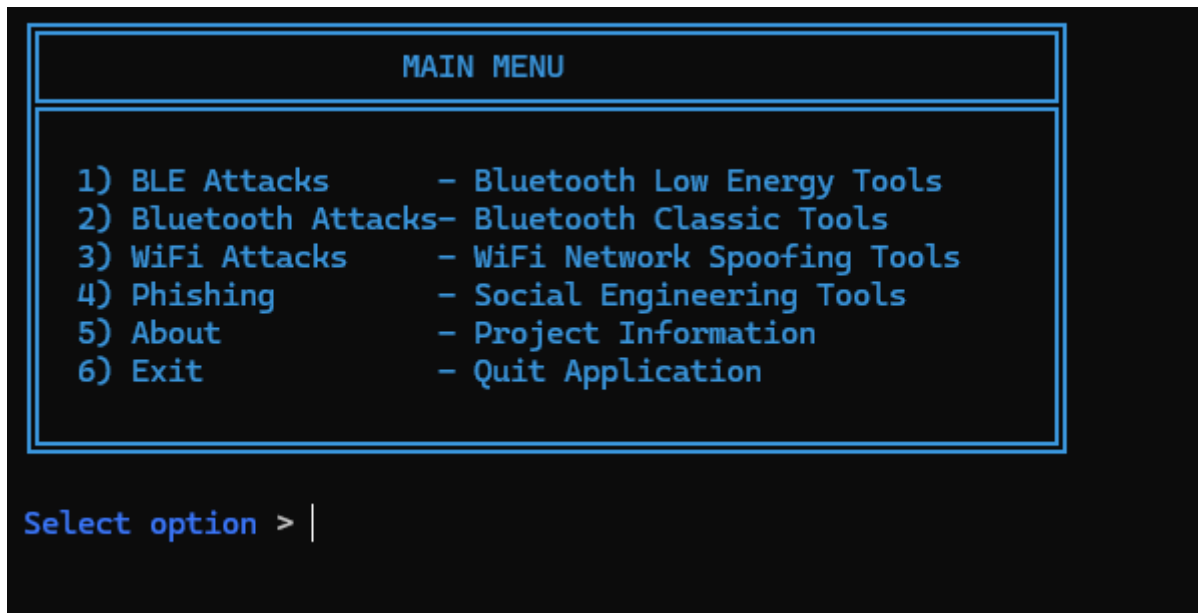


Figure 5.1. The application main menu displayed in the terminal

5.2 Attack Implementation Highlights

Rather than describing all 16 attacks in equal detail, this section highlights the most technically interesting implementations from each domain. Full source code is available in the repository.

WiFi, Deauthentication Attack. This attack targets the fundamental vulnerability of unprotected WiFi management frames. The implementation builds a `mdk4` command with the user's chosen target (SSID, BSSID, or specific device MAC) and executes it as a subprocess, streaming output line-by-line. I implemented three targeting modes: disconnect all devices from a network by name, disconnect all devices from a specific router, or disconnect one specific device. In testing on my own WPA2 network, the attack reliably disconnected devices, which then automatically reconnected once the attack stopped. This demonstrated that most modern home routers still do not enable Protected Management Frames by default.

WiFi, Local Network DoS. This was the most dramatic attack in testing. The module first runs `arp-scan` to discover all devices on the local network, presents them in a list, and lets the user select a target. It then uses `hping3` to send a continuous stream of SYN packets to the target on a random port. When I tested this against my own iPhone, the device froze completely within seconds, buttons stopped responding, video stopped playing, and the screen became unresponsive. Normal operation resumed immediately when the attack was stopped. The effectiveness comes from the minimal network latency on a local network, allowing packets to arrive faster than the device can process them.

BLE, Device Scanner. The scanner uses bleak's `async BleakScanner` with a detection callback that fires for each received BLE advertisement. For each device, the scanner records

MAC address, device name, RSSI (signal strength), and manufacturer data. It looks up the manufacturer using a YAML database of Bluetooth SIG company identifiers, and estimates physical distance from signal strength using a path-loss propagation model: $\text{distance} = 10^{((\text{txPower} - \text{rssi}) / (10 \times n))}$ where n is the path-loss exponent (typically 2 for indoor environments). The display updates continuously, showing a real-time table of all discovered devices with colour-coded signal strength.

BLE DEVICE SCANNER								
#	MAC Address	Name	RSSI	TX Pwr	Dist(m)	Company	First Seen	Last Seen
1	01:3B:C3:8D:F3:13	01-3B-C3-8D-F3-13	-50 dBm	N/A	N/A	Microsoft	19:12:40	19:13:00
2	28:07:08:98:23:1D	28-07-08-98-23-1D	-87 dBm	N/A	N/A	Samsung Electronics ...	19:12:40	19:12:46
3	3E:4A:13:8F:0D:84	3E-4A-13-8F-0D-84	-73 dBm	N/A	N/A	Microsoft	19:12:40	19:13:00
4	40:B3:FA:71:BF:72	AirPods Pro	-57 dBm	N/A	N/A	Apple, Inc.	19:12:48	19:13:00
5	42:74:4E:25:CD:41	42-74-4E-25-CD-41	-92 dBm	7 dBm	17782.8	Apple, Inc.	19:12:41	19:12:59
6	49:2A:5E:BA:95:74	49-2A-5E-BA-95-74	-47 dBm	11 dBm	63.1	Apple, Inc.	19:12:40	19:13:00
7	55:F2:60:BA:B3:77	JBL Go 4	-42 dBm	N/A	N/A	Harman International...	19:12:40	19:13:00
8	55:F2:61:BB:B3:77	55-F2-61-BB-B3-77	-59 dBm	-22 dBm	70.8	Unknown	19:12:40	19:13:00
9	57:5A:29:3A:79:11	57-5A-29-3A-79-11	-97 dBm	N/A	N/A	Apple, Inc.	19:12:41	19:12:55
10	5E:1D:15:1D:99:FD	5E-1D-15-1D-99-FD	-88 dBm	17 dBm	3548.1	Apple, Inc.	19:12:40	19:12:59
11	74:2A:B9:1A:8E:72	74-2A-B9-1A-8E-72	-55 dBm	N/A	N/A	Apple, Inc.	19:12:44	19:12:48
12	78:05:41:4C:CE:F8	GV50CEU_862096072102	-102 dBm	N/A	N/A	Unknown	19:12:51	19:12:51
13	79:27:70:C0:A4:39	79-27-70-C0-A4-39	-53 dBm	N/A	N/A	Apple, Inc.	19:12:46	19:12:48
14	7E:4D:2F:74:1B:C5	7E-4D-2F-74-1B-C5	-36 dBm	N/A	N/A	Apple, Inc.	19:12:41	19:12:59
15	C1:64:15:9C:97:CA	C1-64-15-9C-97-CA	-75 dBm	N/A	N/A	Apple, Inc.	19:12:55	19:12:55
16	C8:12:0B:DD:D5:1D	43" Crystal UHD	-76 dBm	N/A	N/A	Samsung Electronics ...	19:12:40	19:13:00
17	D5:3C:B9:84:DE:C7	D5-3C-B9-84-DE-C7	-52 dBm	N/A	N/A	Apple, Inc.	19:12:40	19:13:00
18	E3:70:9F:5F:D8:41	E3-70-9F-5F-D8-41	-50 dBm	N/A	N/A	Apple, Inc.	19:12:44	19:12:47
19	E4:F9:9F:9D:B6:51	E4-F9-9F-9D-B6-51	-93 dBm	N/A	N/A	Apple, Inc.	19:12:44	19:12:52
20	EF:2C:AF:01:29:F5	EF-2C-AF-01-29-F5	-97 dBm	N/A	N/A	Apple, Inc.	19:12:43	19:12:47
21	F8:BF:82:9A:9F:A1	F8-BF-82-9A-9F-A1	-58 dBm	N/A	N/A	Apple, Inc.	19:12:44	19:13:00
22	FB:47:C8:1C:A3:94	FB-47-C8-1C-A3-94	-88 dBm	N/A	N/A	Apple, Inc.	19:12:41	19:12:59
23	FD:1F:3E:24:D1:5E	FD-1F-3E-24-D1-5E	-35 dBm	N/A	N/A	Apple, Inc.	19:12:43	19:12:59
Total devices: 23								

Figure 5.2. Real-time output of the BLE device scanner

BLE, AirPods Spam. This attack broadcasts fake Apple AirPods advertisements using Apple's manufacturer ID (0x004C) with the correct proximity pairing payload format. Each advertisement includes randomised battery levels for the left ear, right ear, and charging case to make packets unique and prevent the target iPhone from filtering duplicates. The implementation manipulates the Bluetooth adapter's advertising data directly through HCI (Host Controller Interface) socket operations, cycling through different AirPods models. Screenshots of the resulting iOS popup notifications triggered by this attack and the other BLE advertisement spoofing modules are provided in Annex C.

Phishing, Facebook Login. The phishing server hosts a replica of the Facebook login page using Python's `http.server` module. When a GET request arrives, it serves the fake HTML page. When a POST request arrives (form submission), it parses the username and password from the form data, logs them to a file with timestamp and client IP, and returns an HTTP 302 redirect to `https://www.facebook.com`. For public accessibility, the module launches `cloudflared tunnel` as a subprocess, which generates a public HTTPS URL (e.g., `https://something.trycloudflare.com`), no port forwarding, firewall configuration, or SSL certificates required.

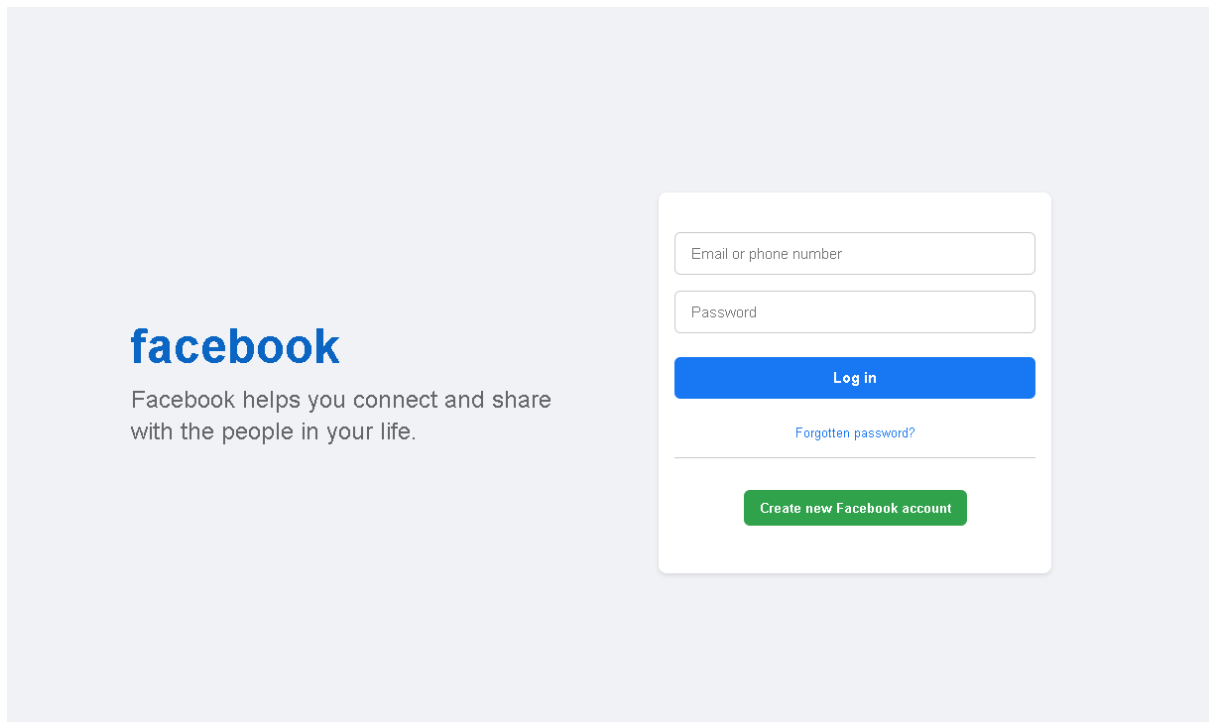


Figure 5.3. Fake Facebook login page

WiFi, ESSID Bruteforce. Hidden WiFi networks do not include their SSID in beacon frames, so they appear in network lists as "Hidden Network." They are not truly hidden, however: when a device that previously connected to the network enters range, it sends probe requests containing the full SSID, which any monitor-mode adapter can capture. My implementation takes a wordlist of common SSIDs and sends probe requests for each entry using `mdk4`'s probe mode, listening for responses. A network that replies confirms its SSID. This attack demonstrates an important security lesson: hiding an SSID is not a security control, it only makes the network less convenient to use while providing no protection against a passive observer with a monitor-mode adapter.

BLE, Name Spoofer. The name spoofer cycles the Bluetooth adapter's advertised device name through a list of common device names (smartphones, headphones, laptops) at a configurable interval. It achieves this using the `btmgmt` command-line tool, specifically the `name` subcommand, which instructs the BlueZ Bluetooth daemon to update the device name in all subsequent advertisements. On target devices, this creates a stream of "new device discovered" notifications, each with a different name but the same underlying MAC address. The attack demonstrates how Bluetooth's discovery model, built around human-readable names, can be manipulated, and it also illustrates that MAC addresses are the real identifiers in the protocol, not names.

Bluetooth Classic, L2CAP DoS. The L2CAP (Logical Link Control and Adaptation Protocol) layer handles connection setup for Bluetooth Classic. When a device receives an L2CAP connection request, it allocates a connection slot and reserves resources to manage it.

The implementation calls `l2ping` as a subprocess with the flood flag (`-f`), directing a continuous stream of L2CAP echo requests at the target's Bluetooth MAC address. Multiple adapters can run in parallel threads, each flooding independently, following the same subprocess orchestration pattern used by the WiFi attacks. `l2ping` is part of the BlueZ package and requires no pairing with the target. The target device must acknowledge each echo request, consuming CPU and connection-handling resources. In testing, this caused noticeable degradation in audio quality on a connected Bluetooth speaker within a few seconds. Modern Bluetooth stacks mitigate this with rate limiting: after a threshold number of requests, the device stops responding. Observing this rate-limiting behaviour during testing is itself educational, showing how protocols evolve to address known attack patterns.

Testing Methodology. All attacks were validated by executing them against devices and networks owned by the author in a controlled home environment. For WiFi attacks, the test network used WPA2 with Protected Management Frames disabled, which reflects the configuration of most consumer routers. For BLE attacks, the target devices were an iPhone and an Android phone belonging to the author. For the L2CAP DoS, the target was a Bluetooth speaker. For phishing, the test used a secondary browser session on the same machine. Each attack was run multiple times to confirm reproducible results. No third-party networks or devices were used at any point during development or testing.

5.3 Key Technical Decisions

Three architectural decisions shaped how the platform was built:

Subprocess over native Python for WiFi. I chose to call external tools (`mdk4`, `aircrack-ng`, `hping3`) via `subprocess.Popen()` rather than implementing packet processing in Python. The reasons were practical: these tools are compiled C programs that handle memory and performance efficiently within the Pi's constraints, they are industry-standard and well-tested, and wrapping them teaches students how real security workflows integrate multiple tools. Python implementing raw 802.11 frame injection would have required the `scapy` library, which is significantly more memory-intensive and slower on ARM.

Async for BLE, threading for phishing. BLE scanning is naturally concurrent, multiple devices broadcast simultaneously, and the scanner must process all of them. Python's `asyncio` with `bleak` provides a clean model for this: a single event loop processes advertisements from all devices without blocking. Phishing, on the other hand, uses `threading` because the HTTP server must run continuously in the background while the main thread manages Cloudflare Tunnel output and user interaction. I chose the concurrency model that best fits each domain rather than forcing a single approach.

Dictionary dispatch over if-elif chains. The menu system uses Python dictionaries to map user choices to attack methods (`{'1': self.action_deauth, '2': self.action_beacon, ...}`). This is more scalable than if-elif chains: adding a new attack is one dictionary entry and one method, rather than modifying a growing conditional block. The same pattern is used consistently across all four attack category menus.

6. Flipper Zero vs My Implementation

The Flipper Zero portable hacking device was the primary inspiration for this project. This chapter compares the two platforms to show where they overlap, where each has advantages, and why the differences exist. The following table provides a detailed feature-by-feature comparison:

Table 6.1. Feature-by-feature comparison between the Flipper Zero and the proposed platform

Feature	Flipper Zero	My Platform	Notes
WiFi			
Beacon Broadcasting	✓	✓	Both use 802.11 frame injection
Deauthentication	✓	✓	Mine supports SSID, BSSID, and device targeting
Network Scanning	✓	✓	Mine uses airodump-ng
AP Network Flood	✗	✓	Mass fake networks from wordlist
Packet Capture	✗	✓	.cap files for Wireshark
Local Network DoS	✗	✓	ARP discovery + SYN flooding
Bluetooth			
BLE Device Scanner	✓	✓	Mine adds company lookup, distance estimation
BLE Spam (iOS)	✓	✓	AirPods and Apple device spam
BLE Spam (Android)	✓	✓	Google/Samsung device spam
BLE Name Spoofing	✗	✓	Rapid adapter name rotation
L2CAP DoS	✗	✓	Bluetooth Classic connection flooding
Phishing			
Fake Login Pages	✗	✓	Facebook and Google replicas
Credential Capture	✗	✓	Logging + Cloudflare Tunnel
Other Modules			
Sub-GHz Radio	✓	✗	No affordable SDR hardware for Pi (\$300+)
Infrared Control	✓	✗	Lack of suitable affordable hardware modules
NFC	✓	✗	Lack of suitable affordable hardware modules

Feature	Flipper Zero	My Platform	Notes
125 kHz RFID	✓	✗	Lack of suitable affordable hardware modules
BadUSB (Keystroke Injection)	✓	✗	Requires additional hardware interface configuration
Platform			
Source Code Access	Partial	Full	Community firmware exists for Flipper
Cost	~\$300+	~\$35	Pi + WiFi adapter
OS	Custom firmware	Full Linux	Linux provides full flexibility
Extensibility	Limited	Easy	Python modules, simple pattern

Shared capabilities. Both platforms support WiFi beacon broadcasting, deauthentication attacks, and network scanning. Both can scan for BLE devices and broadcast fake BLE advertisements that trigger pairing popups on iOS and Android devices. Both provide a unified interface for accessing multiple attack types, the Flipper Zero through a physical screen with a D-pad, my platform through a colour-coded terminal menu.

Where my platform goes further. My platform includes several capabilities the Flipper Zero lacks: local network DoS (ARP discovery + SYN flooding that froze a test iPhone within seconds), HTTP DoS (multi-threaded request flooding), phishing simulation (Facebook and Google login replicas with Cloudflare Tunnel support), ESSID bruteforce (hidden network discovery), comprehensive BLE device scanning with manufacturer identification and distance estimation, Bluetooth Classic L2CAP DoS, and WiFi packet capture for Wireshark analysis. Most importantly, every attack is a readable Python file: students can inspect exactly how a deauthentication command is constructed or how a BLE advertisement is spoofed. The Flipper Zero's official firmware does not expose this level of transparency; users can trigger an attack from the menu but cannot read the code that implements it.

Where the Flipper Zero goes further. The Flipper Zero's most distinctive feature is its CC1101 Sub-GHz transceiver, enabling interaction with garage door openers, car key fobs, and remote controls operating below 1 GHz. It also includes infrared transmission/reception, NFC tag reading and emulation, and 125 kHz RFID card emulation. My platform does not implement these modules because I was unable to find suitable and affordable hardware components. Entry-level SDR options for Sub-GHz cost considerably more (tripling the platform cost), and similarly, compatible hardware modules for infrared, NFC, and RFID that integrate cleanly with the Raspberry Pi setup were not found within the project's scope.

Open-source extensibility. A less obvious but equally important advantage is that the open-source nature of my platform enables it to grow. A lecturer can write a new Python attack module, add one line to the menu dictionary, and the platform gains a new capability in an afternoon. The same process that experienced developers can follow in minutes is also readable and learnable by students, so the act of writing a new attack module is itself a learning exercise. The Flipper Zero's firmware update process is controlled by the manufacturer, and while community firmware alternatives exist, they operate under constraints that a fully open Linux environment does not have.

Analysis. The platforms target fundamentally different use cases. The Flipper Zero excels at portability, physical-world interaction (Sub-GHz, IR, NFC, RFID), and polished user experience. It is a pocket-sized multi-tool for security professionals who need to test devices on the go. My platform excels at educational transparency, affordability, and WiFi/Bluetooth attack depth. The platforms are complementary rather than competitive: a cybersecurity curriculum could use my platform for teaching fundamentals where code transparency and cost matter, and the Flipper Zero for advanced demonstrations where portability and additional radio modules are needed. The key insight is that for educational environments where understanding is more important than convenience, open-source transparency and affordability make my platform a more effective choice.

7. Conclusions and Future Work

This thesis presented an open-source security research platform running on a Raspberry Pi Zero 2 W that demonstrates 16 attacks across WiFi, Bluetooth Low Energy, Bluetooth Classic, and phishing domains. The platform achieves all six objectives set out in the introduction: it unifies multiple security tools behind a single menu-driven interface, demonstrates real protocol vulnerabilities with observable effects, provides an intuitive colour-coded terminal UI, implements a modular architecture where each attack is an independent Python module, offers full source code transparency for educational inspection, and is documented thoroughly for reproducibility.

At approximately \$35 in hardware, any student can build their own station by following the setup guide and cloning the repository, making the platform genuinely accessible rather than just theoretically open. Every attack is implemented in readable Python that students can inspect, modify, and extend, addressing the transparency problem that closed-source tools cannot solve. Testing confirmed that the attacks produce real, observable effects on actual hardware: devices disconnected from WiFi, iOS pairing popups appeared in response to BLE spam, an iPhone froze under local network DoS, and phishing pages successfully captured submitted credentials.

The main limitations are hardware-driven. The Pi's 512 MB RAM prevents memory-intensive operations like GPU-accelerated password cracking. The single external WiFi adapter cannot maintain internet connectivity while in monitor mode, which limits simultaneous operations. CPU performance on the ARM Cortex-A53 constrains real-time packet processing throughput. Some attacks are also mitigated by modern defences (WPA3 Protected Management Frames blocks deauthentication spoofing, and Bluetooth rate limiting curtails the L2CAP DoS), but this is itself an educational lesson: security is an evolving discipline where defences continuously improve in response to known attack patterns.

Several directions could meaningfully extend this platform:

- **Handshake cracking offload:** implement a workflow that captures WPA2 four-way handshakes on the Pi and exports the capture file to a desktop machine running hashcat with GPU acceleration. This would complete the WiFi password security education story without requiring hardware the Pi cannot support, and would mirror the workflow that professional penetration testers use in practice.
- **Sub-GHz module:** if an affordable SDR (Software-Defined Radio) module becomes available that is compatible with the Raspberry Pi at a price point below \$50, adding Sub-GHz capabilities would match the Flipper Zero's most distinctive feature. The

CC1101 transceiver operating below 1 GHz is the main area where my platform currently falls short.

- **Zigbee and IoT protocol support:** adding an IEEE 802.15.4 transceiver such as the CC2531 USB dongle would allow the platform to sniff Zigbee traffic used in smart home devices. As IoT deployments grow, Zigbee security analysis is becoming an increasingly relevant skill for network administrators and security researchers.
- **Curriculum integration:** structured lab exercises with explicit learning objectives, expected outputs, and assessment criteria would allow this platform to be adopted into formal cybersecurity courses without requiring the instructor to design the pedagogical framework around it. Each of the 16 attacks maps naturally to a distinct concept (frame forgery, advertisement spoofing, credential harvesting) that can be assessed independently.

References and Bibliography

- [1] FLOCK4H, "Jammy: Security attack orchestration platform," GitHub, 2023. [Online]. Available: <https://github.com/FLOCK4H/Jammy>.
- [2] Skittleson, "Bluetooth-WOS: Bluetooth attack and scanning tools," GitHub. [Online]. Available: <https://github.com/skittleson/bluetooth-wos>.
- [3] Bhaviktutorials, "Shark: Network reconnaissance framework," GitHub. [Online]. Available: <https://github.com/Bhaviktutorials/shark>.
- [4] SwitchDoc Labs, "iBeacon-Scanner: Bluetooth Low Energy scanner implementation," GitHub. [Online]. Available: <https://github.com/switchdoclabs/iBeacon-Scanner->.
- [5] RapierXbox, "ESP32 Sour Apple: Apple BLE advertisement spoofing via proximity notification frames," GitHub. [Online]. Available: <https://github.com/RapierXbox/ESP32-Sour-Apple>.
- [6] The Aircrack-ng Project, "Aircrack-ng: Wi-Fi security auditing tools suite," 2024. [Online]. Available: <https://www.aircrack-ng.org/>.
- [7] Aircrack-ng, "mdk4: Wi-Fi pentesting framework," GitHub, 2024. [Online]. Available: <https://github.com/aircrack-ng/mdk4>.
- [8] H. Blidh et al., "Bleak: Bluetooth Low Energy platform agnostic client for Python," GitHub, 2024. [Online]. Available: <https://github.com/hbldh/bleak>.
- [9] The BlueZ Project, "BlueZ: Official Linux Bluetooth protocol stack," 2024. [Online]. Available: <http://www.bluez.org/>.
- [10] morrownr, "8821au-20210708: Linux driver for USB Wi-Fi adapters based on the Realtek RTL8821AU chipset," GitHub. [Online]. Available: <https://github.com/morrownr/8821au-20210708>.
- [11] S. Antirez, "hping3: Active network smashing tool," GitHub. [Online]. Available: <https://github.com/antirez/hping>.
- [12] R. Hill, "arp-scan: ARP scanning and fingerprinting tool," GitHub. [Online]. Available: <https://github.com/royhills/arp-scan>.
- [13] Cloudflare, Inc., "Cloudflare Tunnel: Secure outbound connections to Cloudflare's edge network," Cloudflare Developer Documentation, 2024. [Online]. Available: <https://developers.cloudflare.com/cloudflare-one/connections/connect-networks/>.
- [14] IEEE, "IEEE Standard for Information Technology—Telecommunications and Information Exchange Between Systems Local and Metropolitan Area Networks—Specific Requirements Part 11: Wireless LAN Medium Access Control (MAC) and

- Physical Layer (PHY) Specifications," IEEE Std 802.11-2020, Feb. 2021, doi: 10.1109/IEEESTD.2021.9363693.
- [15] Bluetooth Special Interest Group, "Bluetooth Core Specification," Version 5.4, Bluetooth SIG, Kirkland, WA, USA, Feb. 2023. [Online]. Available: <https://www.bluetooth.com/specifications/specs/core-specification-5-4/>.
- [16] Bluetooth Special Interest Group, "Assigned Numbers: Company Identifiers," 2024. [Online]. Available: <https://www.bluetooth.com/specifications/assigned-numbers/>.
- [17] N. Borisov, I. Goldberg, and D. Wagner, "Intercepting mobile communications: The insecurity of 802.11," in *Proc. 7th Annual ACM International Conference on Mobile Computing and Networking (MobiCom '01)*, Rome, Italy, Jul. 2001, pp. 180–189, doi: 10.1145/381677.381695.
- [18] M. Vanhoef and F. Piessens, "Key reinstallation attacks: Forcing nonce reuse in WPA2," in *Proc. 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS '17)*, Dallas, TX, USA, Oct.–Nov. 2017, pp. 1313–1328, doi: 10.1145/3133956.3134027.
- [19] A. M. Lonzetta, P. Cope, J. Campbell, B. J. Mohd, and T. Hayajneh, "Security vulnerabilities in Bluetooth technology as used in IoT," *Journal of Sensor and Actuator Networks*, vol. 7, no. 3, p. 28, Jul. 2018, doi: 10.3390/jsan7030028.
- [20] T. S. Rappaport, *Wireless Communications: Principles and Practice*, 2nd ed. Upper Saddle River, NJ: Prentice Hall, 2002.
- [21] Raspberry Pi Foundation, "Raspberry Pi Zero 2 W product brief," Raspberry Pi Foundation, Cambridge, UK, 2021. [Online]. Available: <https://www.raspberrypi.com/products/raspberry-pi-zero-2-w/>.
- [22] Flipper Devices Inc., "Flipper Zero: Multi-tool device for hackers," 2023. [Online]. Available: <https://flipperzero.one/>.

Annex A: Hardware Setup and Installation Guide

Hardware Used

Table A.1. Hardware components and approximate costs

Component	Model	Purpose	Cost
Computer	Raspberry Pi Zero 2 W	Main platform	~\$15
WiFi Adapter	TP-Link Archer T2U PLUS	Monitor mode, packet injection	~\$20
Storage	32 GB MicroSD card	OS and data	~\$5
USB-A to Micro USB Adapter	Generic USB-A → Micro USB OTG adapter	Connect the USB-A WiFi adapter to the Pi's Micro USB port	~\$2
Micro USB to USB-C Adapter	Generic Micro USB → USB-C adapter	Power and connect the Pi to a laptop via USB-C cable	~\$2

Installation Steps

Step 1: Flash Raspberry Pi OS Lite to MicroSD using Raspberry Pi Imager. Enable SSH and configure WiFi in imager settings.

Step 2: Install system packages

```
sudo apt-get update && sudo apt-get upgrade -y
sudo apt-get install -y python3 python3-pip python3-dev git curl wget
sudo apt-get install -y bluez bluez-tools python3-bluez python3-dbus
libbluetooth-dev
sudo apt-get install -y aircrack-ng mdk4 wifite
sudo apt-get install -y dnsmasq hostapd iptables arp-scan hping3
sudo apt install -y python3-bleak python3-yaml
```

Step 3: Install WiFi adapter driver

```
sudo apt-get install -y build-essential dkms bc linux-headers-generic
cd ~ && git clone https://github.com/morrownr/8821au-20210708.git
cd 8821au-20210708 && sudo ./install-driver.sh
sudo reboot
```

Step 4: Enable monitor mode

```
sudo nmcli device set wlan1 managed no
sudo ip link set wlan1 down
sudo iw dev wlan1 set type monitor
sudo ip link set wlan1 up
iwconfig wlan1 # Should show Mode:Monitor
```

Step 5: Initialise Bluetooth

```
sudo rfkill unblock bluetooth
sudo hciconfig hci0 reset
sudo btmgmt power on
sudo btmgmt connectable on
sudo btmgmt discov on
```

Step 6 (optional): Install Cloudflare Tunnel for phishing

```
wget
https://github.com/cloudflare/cloudflared/releases/download/2024.12.0/cloudflared-linux-arm64
chmod +x cloudflared-linux-arm64
sudo mv cloudflared-linux-arm64 /usr/local/bin/cloudflared
```

Annex B: Ethical and Legal Framework for Security Testing

The attacks demonstrated in this platform — deauthentication, BLE advertisement spoofing, credential phishing, and network flooding — are techniques that exist in real-world offensive security. Using them without authorisation on systems or networks owned by others is illegal in most jurisdictions. This annex documents the legal framework applicable to this project and defines the conditions under which the platform may be used ethically.

Legal Framework

European Union — Directive 2013/40/EU on attacks against information systems establishes minimum criminal penalties across EU member states for illegal access to information systems, illegal system interference, and illegal data interception. Under this directive, sending deauthentication frames to a WiFi network you do not own, intercepting Bluetooth communications from a device you do not control, or hosting phishing pages targeting real users would constitute criminal offences subject to imprisonment and fines in all EU member states, including Romania.

Romania — Law No. 161/2003 on certain measures to ensure transparency in public dignities, public functions, and the business environment, prevention and sanctioning of corruption (Title III — Cybercrime) transposes the EU Cybercrime Convention into Romanian law. Articles 42–44 specifically criminalise unauthorised access to computer systems, illegal interception of computer data, and data or system interference. A WiFi deauthentication attack targeting a neighbour's network or an employer's access point would fall under Article 44 (system interference); credential harvesting from real users via phishing would fall under Articles 42 and 43.

Conditions for Authorised Use

The platform may be used legally and ethically under the following conditions:

- **Owned hardware and networks:** testing is performed exclusively on WiFi networks, Bluetooth devices, and computing equipment owned by the person conducting the test. This is the mode of use followed throughout this thesis.
- **Written permission:** for testing on third-party infrastructure (a company's network, a client's devices), explicit written authorisation from the system owner must be obtained in advance, specifying the scope, duration, and methods permitted. This is standard practice in professional penetration testing engagements.

- **Controlled laboratory environment:** in an academic setting, the institution must designate an isolated network segment or physical laboratory for security testing, and students must be informed that testing is restricted to that environment. Attacks must never reach production networks or devices belonging to other students or staff.
- **No credential retention:** the platform's phishing modules log captured credentials to a local file for demonstration purposes. In any educational exercise, these files must be deleted immediately after the session and must never be used to access actual accounts.
- **Responsible disclosure:** if a student discovers a previously unreported vulnerability using this platform, standard coordinated disclosure norms apply — notify the vendor privately with a technical description, allow a 90-day remediation period, and publish findings only afterwards.

Ethical Design Decisions in the Platform

Several design choices in the platform reflect responsible security tool development:

- All attacks terminate immediately when the user presses Ctrl+C, ensuring there is no mechanism for persistent or background operation once the operator stops the session.
- The phishing modules redirect the victim to the real website after credential capture, a design choice that minimises disruption and makes the credential capture event observable only to the operator who is watching the terminal, not persistent in any external system.
- No credentials, packet captures, or discovered device data are transmitted outside the local machine. All output stays local unless the operator explicitly exports it.
- The platform includes no exploit code targeting specific software vulnerabilities (CVEs). All techniques demonstrated operate at the protocol level and are mitigated by standard configuration choices (enabling WPA3 PMF, keeping Bluetooth firmware updated), reinforcing the defensive lessons alongside the offensive demonstrations.

Annex C: BLE iOS Advertisement Screenshots

The following screenshots were captured on my iPhone during testing of the BLE advertisement spoofing attacks. It demonstrates how the operating system responds to forged BLE advertisements, displaying native pairing and notification popups indistinguishable from those triggered by genuine Apple devices.

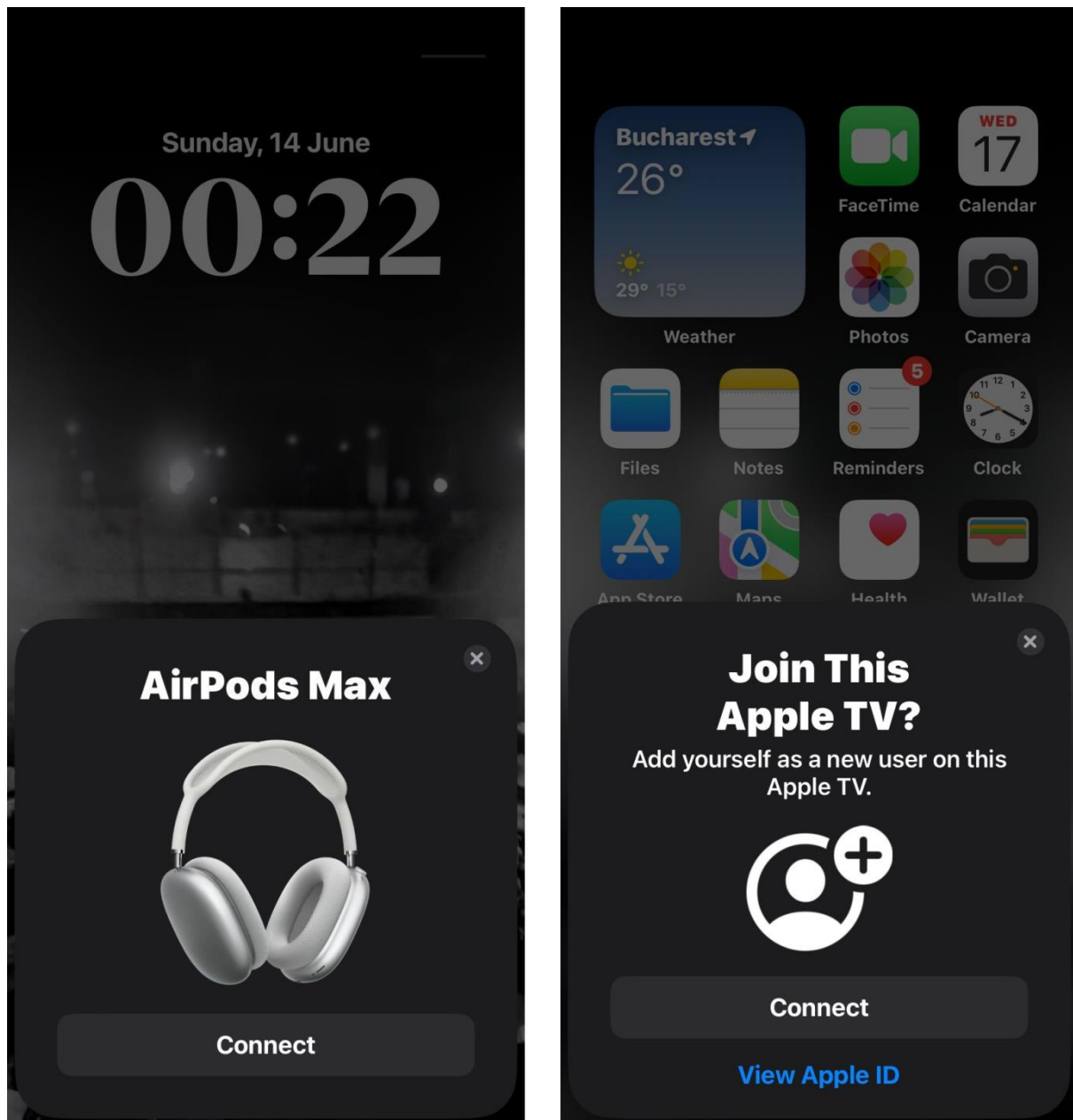


Figure C.1. iOS BLE advertisement popups: AirPods pairing notification (left) and Apple device advertisement popup (right)

Annex D: List of Figures

Figure 3.1. Complete hardware setup	9
Figure 4.1. System architecture: three-layer component view	13
Figure 5.1. The application main menu displayed in the terminal	16
Figure 5.2. Real-time output of the BLE device scanner	17
Figure 5.3. Fake Facebook login page	18